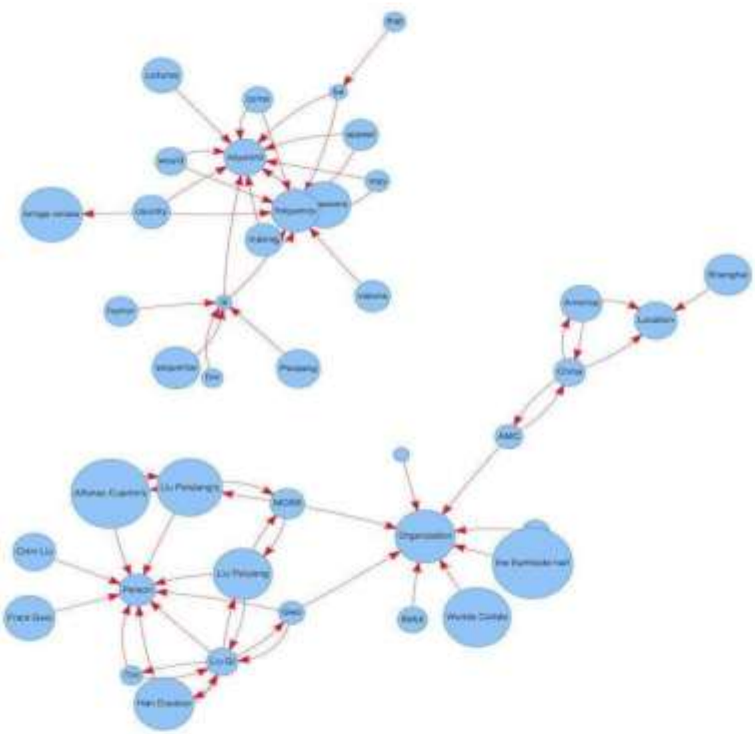
	Marwadi University **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No : 92301733009**

Aim: TextRank for Document Summarization

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.

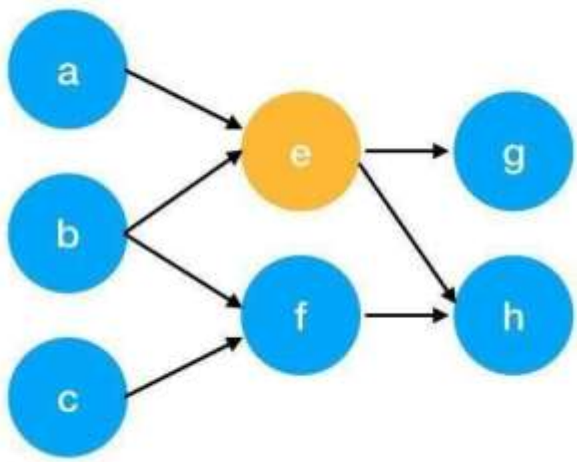


Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}\,, j \in \{a, b\}$$

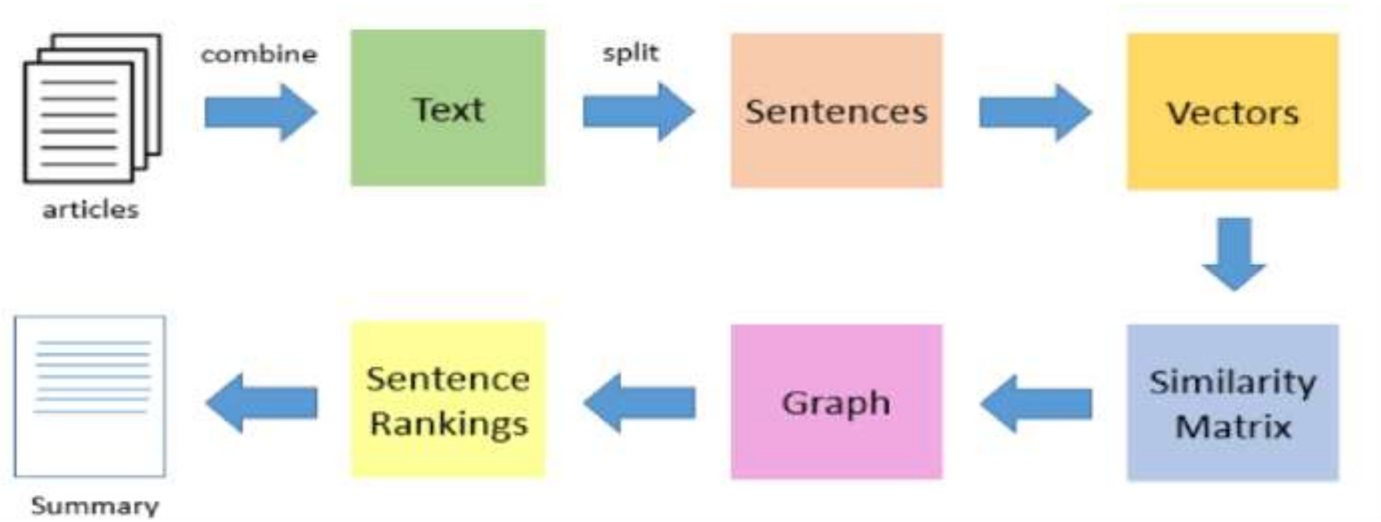
$$\begin{aligned}\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\ &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\ &= S(V_a) + \frac{1}{2} S(V_b)\end{aligned}$$

We can get the weight of webpage e by the following function.

$$S(V_e) = (1-d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

TextRank for document Summarization



TextRank works in the following steps:
1. Tokenize documents into sentences.
2. Preprocess each sentence in the document.
3. Count key phrases and normalize them or produce TFIDF Matrix, you can also use any kind of vectorization such as spacy vectors.
4. Calculate the Jaccard Similarity between sentences and key phrases.
5. Rank the sentences with higher significance.

Pre Lab Exercise:

1. How keyword extraction is similar to document summarization process?
   - Both aim to identify important information from the text.
   - Both use TextRank algorithm (graph-based approach).
   - Both involve text preprocessing like tokenization and stop-word removal.
   - Both rank elements using importance scores (PageRank concept).
   - Both help in reducing large text into concise form.

2. How keyword extraction is different to document summarization process?
   - Keyword Extraction:
     - Extracts important words or phrases only
     - Output is a list of keywords
     - Focus is on terms
   - Document Summarization:
     - Extracts important sentences
     - Output is a meaningful paragraph/summary
     - Focus is on overall context and readability

3. Limitation of TextRank as a summarizer
   - No deep understanding of meaning (only statistical relationships)
   - May miss important sentences that are contextually important
   - Produces incoherent summaries sometimes (lack of flow)
   - Depends heavily on similarity measure
   - Ignores semantic relationships and tone
   - Not suitable for very complex or technical documents

Program (Code):

```
#importing basic libraries
import numpy as np
import nltk
import networkx as nx
from nltk.tokenize import sent_tokenize,word_tokenize
from nltk.corpus import stopwords
from sklearn.metrics.pairwise import cosine_similarity
from collections import Counter
#download nltk resources
nltk.download('punkt')
nltk.download('stopwords')
```

```
def preprocess_text(text):
  stop_words = set(stopwords.words('english'))
  sentences=sent_tokenize(text)
  word_frequencies = []
  for sent in sentences:
    words = word_tokenize(sent.lower())
    filtered_words = [w for w in words if w.isalnum() and w not in stop_words]
    word_frequencies=Counter(filtered_words)
  return sentences,word_frequencies
def similarity_matrix(word_frequencies):
  size=len(word_frequencies)
  sm=np.zeros((size,size))
  for i in range(size):
    for j in range(size):
      if i!=j:
        words1=word_frequencies[i]
        words2=word_frequencies[j]
        common_words=set(words1.keys()).union(set(words2.keys()))

        vec1 = np.array([words1[word] for word in common_words])
        vec2 = np.array([words2[word] for word in common_words])

        sm[i][j]=cosine_similarity([vec1],[vec2])[0,0]
      return sm
def textrank(text,num=3):
  sentences,word_frequencies=preprocess_text(text)
  sm=similarity_matrix(word_frequencies)

  #build graph and apply pagerank

  graph=nx.from_numpy_array(sm)
  scores=nx.pagerank(graph)

  ranked_sentences=sorted(((scores.get(i,0),s)
          for i,s in enumerate(sentences)),reverse=True)
  summary=" ".join(sent for _,sent in ranked_sentences[:num])
  return summary
```

text = """India, officially the Republic of India,[j][20] is a country in South Asia. It is the seventh-largest country by area; the most populous country since 2023;[21] and, since its independence in 1947, the world's most populous democracy.[22][23][24] Bounded by the Indian Ocean on the south, the Arabian Sea on the southwest, and the Bay of Bengal on the southeast, it shares land borders with Pakistan to the west;[k] China, Nepal, and Bhutan to the north; and Bangladesh and Myanmar to the east. In the Indian Ocean, India is near Sri Lanka and the Maldives; its Andaman and Nicobar Islands share a maritime border with Myanmar, Thailand, and Indonesia.

Modern humans arrived on the Indian subcontinent from Africa no later than 55,000 years ago.[26][27][28] Their long occupation, predominantly in isolation as hunter-gatherers, has made the region highly diverse.[29] Settled life emerged on the subcontinent in the western margins of the Indus river basin 9,000 years ago, evolving gradually into the Indus Valley Civilisation of the third millennium BCE.[30] By 1200 BCE, an archaic form of Sanskrit, an Indo-European language, had diffused into India from the northwest.[31][32] Its hymns recorded the early dawnings of Hinduism in India.[33] India's pre-existing Dravidian languages were supplanted in the northern regions.[34] By 400 BCE, caste had emerged within Hinduism,[35] and Buddhism and Jainism had arisen, proclaiming social orders unlinked to heredity.[36] Early political consolidations gave rise to the loose-knit Maurya and Gupta Empires.[37] This era was noted for creativity in art, architecture, and writing,[38] but the status of women declined,[39] and untouchability became an organised belief.[l][40] In South India, the Middle kingdoms exported Dravidian language scripts and religious cultures to the kingdoms of Southeast Asia.[41]"""

```
nltk.download('punkt_tab')
print(textrank(text,num=5))
```

Results:

Observation:

- The TextRank algorithm was applied to generate a summary by ranking sentences based on their importance.
- Sentences were treated as nodes and similarity between them was used to form a graph structure.
- It was observed that sentences with higher connections to other sentences received higher ranks and were included in the summary.
- The generated summary captured the main idea of the document but sometimes lacked proper flow and coherence.
- Some important sentences were not included because TextRank does not understand deep semantic meaning.
- The quality of the summary depended on preprocessing steps and similarity calculation methods.
- Overall, TextRank proved to be an effective unsupervised technique for extractive summarization, but it may require manual refinement for better readability.

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** TextRank for Document Summarization	
Experiment No: 10	**Date:**	**Enrolment No : 92301733009**

Post Lab Exercise:

Comment over the summary obtained. Which sentence you think should be there in your summary, but was not spotted by textrank? Rate the overall summary in the scale of 1-5 (with 1 as least and 5 as highest rating). Paste the code, your portfolio, output and your analysis.

- The summary generated using TextRank successfully captures the main idea of the portfolio, including key areas like Artificial Intelligence, Machine Learning, and Data Analytics.
- The 50% summary includes most of the important details such as skills, projects, and interests, making it informative.
- The 25% summary is more concise and readable but loses some important information due to higher compression.
- The summary is extractive in nature, so it selects sentences directly from the text rather than generating new ones.
- However, the flow of the summary is not always smooth, and some important context is missing.